Indian Institute of Technology Bhubaneswar

School of Electrical Sciences

Autumn Mid-Semester Examination-2021

Subject Name: Compiler Design	**Subject Code: CS4L001**

Date: 22.09.2021

Duration: 2 Hours **Full Marks: 50**

Answer a new section on a fresh page (strictly) and all parts of a section together.

<u>Section A: Answer the following questions briefly: 2+3+3+3+2 +3 = 16 marks.</u>

(A1) Consider the alphabet **Σ = {a, b}**. Define a short (possibly the shortest), regular expression that generates strings over **Σ** that contain *exactly* one "**b**" and *at least* one "**a**".

(A2) Consider the grammar with non-terminals **N = {S, B, C}**, terminals **T= {a, d, i, k, e}**, with **S** as the start symbol, and the following set of rules: **S → iCkSB|a** **B → eS|ϵ** **C → d**
Is the grammar left-recursive? Is it ambiguous? Justify your answer.

(A3) Consider the context-free grammar: **S → a S b S | b S a S | ε**, and consider the string **aabbab**

1. Give a leftmost derivation for the string.
2. Give a parse tree for the string.
3. Is the grammar ambiguous or unambiguous? Justify your answer.

(A4) Consider the following grammar. Eliminate left recursion. Does left recursion elimination make it suitable for top-down parsing?

E → E E + | E E * | a

(A5) Discuss very briefly why the lexical analysis phase of a compiler is implemented separately. Briefly discuss regarding longest match rule in lexical analysis (via a simple example).

(A6) Consider the CFG fragment with non-terminal symbols **{D, T, L}**, with start symbol **D**, terminal symbols {**id**, **:**, **;**, **,**, **var**, **integer**, **real** } and the productions **P** listed below. Is this grammar suitable to be parsed using a predictive **LL(1)** parsing algorithm? Justify/briefly explain why.

D → var L : T ; $

T → integer

T → real

L → L , id

L → id

Section B: Lexical analysis: 2+3+2+2+5 = 14 marks

(B1) A regular expression of a language over *{a, b}* is **(a|(a*b))**. Design a DFA for the language.

(B2) A simple 'maximal-length' tokenizer is designed for the language of (B1). The input to the scanner is a^{10}<eof> (10 a's followed by end-of-file). How many character-read and roll-backs are required to tokenize the considered input?

(B4) Given the following tokens and their associated regular expressions, show what output is produced when this lex scanner is run over the following string: **bbaacacbabcacabbcc**

```
%%
aa*(b|c)*a    { printf(" T1 "); }
bb*(a|c)*b    { printf(" T2 "); }
cc*(b|a)*c    { printf(" T3 "); }
.|\n          {printf(" ? ");  }
%%
```

(B3) Given the following tokens and their associated regular expressions, show what output is produced when this lex scanner is run over the following string: **systemmatic**

```
%%

"system"          { printf(" T1 "); }

"systematic"      { printf(" T2 "); }

.|\n              { printf(" ! ");  }

%%
```

(B5) Consider that in a programming language (to be developed) an **integer constant** contains one or more digits, but it may also contain embedded "#" symbols. Any "#" symbol must be preceded and followed by at least one digit, which implies that a number may not begin or end with an "#" symbol, and numbers may not contain two or more adjacent "#" symbols. Here, "#" symbols have no significance other than readability.
Examples: 5, 14, 017, 5#01#7, 8#149#574.

Integers may also be written in bases other than 10 by preceding the number with the base and a underscore character.
For example, all of the below constants represent the decimal number 15231:
15231, *10_15231*, *16_3B7F*, *16_03B#7F*, *2_00111011#0111111*, and *8_35577*.

We will simplify, and finally consider the following constraints/requirements for this question:

- The *base of a number* (preceding the underscore sign) must be a number that is at least 2 and at most 16, and is written in decimal.
- The *digits in a number* consist of the decimal digits 0 through 9, and the letters A, B, C, D, E, F, written only in upper case.
- If no base is present, the digits may only be 0 through 9. Note that an integer constant may also contain embedded "#" symbols as described above.

(B5.1) Give a regular definition for integer constants as described above.

(B5.2) Model/design a generalized DFA that recognizes integer constants as described in the problem and generated by the regular definition in your answer to part (B5.1).

Section C: Top-down Parsing: 10 marks.

Consider the following grammar with non-terminals {**S, X, Y, Z**}, terminal symbols {**a, b, c, d, (,)**}, and start symbol **S**:

$$S \rightarrow XYZ$$
$$X \rightarrow (S) \mid aSb \mid d$$
$$Y \rightarrow aY \mid \epsilon$$
$$Z \rightarrow cZ \mid \epsilon$$

(C1) Is the above grammar **LL(1)**? Make changes to the above grammar (if necessary) to make it suitable for **LL(1)** parsing.

(C2) ***Briefly*** describe the rules for computing *FIRST* and *FOLLOW*. Compute *FIRST* and *FOLLOW* sets for each non-terminal.

(C3) Compute **LL(1)** parse table.

(**C4**) Show the actions of the **LL(1)** parser given the input string: **(da)ac**

Section D: Bottom-up Parsing (LR(1), LALR(1)): 10 marks

Consider the following grammar with the set of non-terminals **{S, X, Y}**, start symbol **S**, and terminal symbols **{a, b, f, g}**. Construct **LR(1)** automaton, and the **LR(1)** parsing table. Is this grammar **LR(1)**? Derive the corresponding **LALR(1)** automaton and parsing table. Is this grammar **LALR(1)**? Justify your answer.

NOTE: The grammar is augmented with the production "S' → S".

(1) **S' → S**
(2) **S → Xa**
(3) **S → bXf**
(4) **S → Yf**
(5) **S → bYa**
(6) **X → g**
(7) **Y → g**